

Project Report: Servers and Drones

Advanced Algorithms Mini-Project

Group Member(s): Muhammad Omais Rana

Exercise 1:

Geometric Modeling (Doors)

To visualize connectivity, "doors" were rendered on shared edges. The position of a door was calculated using linear interpolation along the edge vector. We modified Polygon::draw to render a white line over the black edge at the calculated midpoint.

Code Snippet (Polygon::draw logic): C++

```
if (N > 0) {  
  
    painter.setPen(QPen(Qt::white, 7));    const  
  
    double doorWidth = 25.0;  
  
  
    for (int i = 0; i < N; i++) {  
  
        Vector2D P1 = tabPts[i];  
  
        Vector2D P2 = tabPts[(i + 1) % N];  
  
        Vector2D V = P2 - P1;  
  
        Vector2D M = P1 + 0.5 * V;  
  
  
        double len = V.length();  
  
        if (len == 0) continue;  
  
        Vector2D u = (1.0 / len) * V;  
  
  
        Vector2D Q0 = M - (doorWidth / 2.0) * u;  
  
        Vector2D Q1 = M + (doorWidth / 2.0) * u;
```

```

        painter.drawLine(QPointF(Q0.x, Q0.y), QPointF(Q1.x, Q1.y));
    }
}

```

Exercise 2: Graph Construction & Pathfinding

A graph was built to represent room connectivity. We iterated through all server pairs; if they shared an identical edge (vertices matched), a Link was created.

Code Snippet (createServersLinks): C++

```

for (int i = 0; i < servers.size(); ++i) {    for
(int j = i + 1; j < servers.size(); ++j) {

Polygon &poly1 = servers[i].area;

    Polygon &poly2 = servers[j].area;

    bool connected = false;

    for (int k = 0; k < poly1.nbVertices(); ++k) {

        Vector2D p1 = poly1[k];

        Vector2D p2 = poly1[(k + 1) % poly1.nbVertices()];

        for (int m = 0; m < poly2.nbVertices(); ++m) {

            Vector2D q1 = poly2[m];

            Vector2D q2 = poly2[(m + 1) % poly2.nbVertices()];

            if ((p1 == q1 && p2 == q2) || (p1 == q2 && p2 == q1)) {

                Link* newLink = new Link(&servers[i], &servers[j], {p1, p2});

                links.append(newLink);          servers[i].links.append(newLink);

                servers[j].links.append(newLink);          connected = true;          break;

```

```

        }
    }
    if (connected) break;
}
}
}

```

Pathfinding Algorithm: We implemented Dijkstra's Algorithm to calculate the shortest path from every server to every other server. The results were stored in a bestDistance table, saving the “*first link*” of the optimal path to allow hop-by-hop navigation.

Code Snippet (fillDistanceArray logic): C++

```

for (int src = 0; src < nServers; ++src) {
    QVector<double> dist(nServers, 1e9);
    QVector<int> parent(nServers, -1);
    QVector<bool> visited(nServers, false);

    dist[src] = 0.0;

    for (int i = 0; i < nServers; ++i) {
        int u = -1;
        double minDist = 1e9;
        for (int j = 0; j < nServers; ++j) {
            if (!visited[j] && dist[j] < minDist) {
                minDist = dist[j];
                u = j;
            }
        }
    }
}

```

```

        if (u == -1 || minDist == 1e9) break;
visited[u] = true;

        for (Link* l : servers[u].links) {

            Server* neighborNode = (l->getNode1()->id == servers[u].id) ? l->getNode2() : l->getNode1();

            int v = neighborNode->id;

            double weight = l->getDistance();
            if (dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                parent[v] = u;
            }
        }
    }

    for (int dest = 0; dest < nServers; ++dest) {
distanceArray[src][dest] = dist[dest];

        if (dest == src) {
            servers[src].bestDistance[dest] = {nullptr, 0.0};
        } else if (dist[dest] >= 1e9) {
servers[src].bestDistance[dest] = {nullptr, -1.0};
        } else {
            int curr = dest;
            while
            (parent[curr] != src && parent[curr] != -1) {
curr = parent[curr];
            }
        }
    }

```

```

        Link* bestLink = nullptr;
for (Link* l : servers[src].links) {
    Server* neighbor = (l->getNode1()->id == servers[src].id) ? l->getNode2() : l->getNode1();
    if (neighbor->id == curr)
    {
        bestLink = l;
        break;
    }
}
servers[src].bestDistance[dest] = {bestLink, dist[dest]};
}
}
}

```

Complexity of the Algorithm: The complexity of the graph creation algorithm is $O(N^2 * M^2)$, where,

- **N** is the number of servers.
- **M** is the average number of edges per Voronoi polygon.

Computed Distances

[Shortest Path Distances]							
S\D	Masyaf	Venice	Constanti	Boston	Paris	London	Alexandri
Masyaf	0	362	213	895	558	776	213
Venice	362	0	562	533	196	414	575
Constanti	213	562	0	999	662	772	426
Boston	895	533	999	0	337	429	1109
Paris	558	196	662	337	0	218	771
London	776	414	772	429	218	0	989
Alexandri	213	575	426	1109	771	989	0

Exercise 3: Drone Logic & Animation

A state machine controls drone movement. First, drones are initialized by identifying their starting server.

Code Snippet (Initialization in MainWindow):

C++

```
for (auto &drone : ui->canvas->drones) {  
    Server* startServer = drone.overflownArea(ui->canvas->servers);    if (startServer) {  
        drone.destination = Vector2D(startServer->position.x(), startServer->position.y());  
    }  
}
```

Movement Logic: The drone logic checks if it has reached a Server or a Door and updates the destination accordingly.

Code Snippet (Drone::move logic): C++

```
if (d < minDistance) {    if  
(!connectedTo) return;  
    Vector2D serverPos(connectedTo->position.x(), connectedTo->position.y());  
  
    if ((destination - serverPos).length() < 1.0) {  
        if (target && connectedTo->id == target->id) {  
            speed = Vector2D(0,0);  
            return;  
        }  
        if (target && target->id < connectedTo->bestDistance.size()) {  
            Link* nextLink = connectedTo->bestDistance[target->id].first;            if  
(nextLink) {  
                destination = nextLink->getEdgeCenter();
```

```

    }
}
else {
    for (Link* link : connectedTo->links) {        Vector2D
doorPos = link->getEdgeCenter();        if ((position -
doorPos).length() < minDistance + 2.0) {
        Server* neighbor = (link->getNode1()->id == connectedTo->id) ? link->getNode2() : link-
>getNode1();
        connectedTo = neighbor;        destination = Vector2D(neighbor-
>position.x(), neighbor->position.y());        break;
    }
}
}
}

```

3. Custom Configuration

I used a custom json file named “*creed.json*” to test the algorithm. The results were satisfactory.